NPU Assignment

Problem - 1

4-request Round Robin Arbiter

4 request inputs;

req[3:0] = req3, req2, req1, req0.

4 grant o/ps;

grant[4:0] = gnt3, gnt2, gnt1, gnt0.

Objective :- i) At any given clock cycle, only one grant signal should be high, corresponding to the active request.

ii) The priority should rotate after every successful grant.

iii) No requester is starved and all get fair access.

Ideation :- We have to design an arbiter that allows access to four different requests to one at a time based on their priority.

~~After~~ Also the priority should change in every clock cycle so that all request input gets equal chance of being at the highest priority in order to avoid starvation.

Note :- The core idea is whoever was just served becomes the lowest priority, and the next requester becomes the highest priority.

0 → 1 → 2 → 3 → 0

Design

Memory :- We have to store which req. i/p was last granted so that we can decide priority for next access.

4 states

- Last 0 (requestor 0 was granted most recently)
- Last 1 (requestor 1 was granted most recently)
- Last 2 (requestor 2 was granted most recently)
- Last 3 (requestor 3 was granted most recently)

Example! Suppose we are in (Last2) state,

then our check order will be 3, 0, 1, 2

The first requestor in that order with req high will be granted.

↳ This rule gives fairness as whoever was just served moves to the back of the line and everyone else moves one step closer to the front.

One hot encoding for the grant output

Because it was mentioned in the problem statement, that at a time only one output bit should be high, rest should be low.

Therefore we are using one-hot encoding for grant o/p.

One-hot encoding represents a value using one wire per possible value where exactly one wire is 1 and rest 0

Diff b/w binary and one hot encoding

value	Binary (2-bits)	One hot (4 bits)
0	00	0001
1	01	0010
2	10	0100
3	11	1000

Binary needs fewer wires as compared to One-hot.

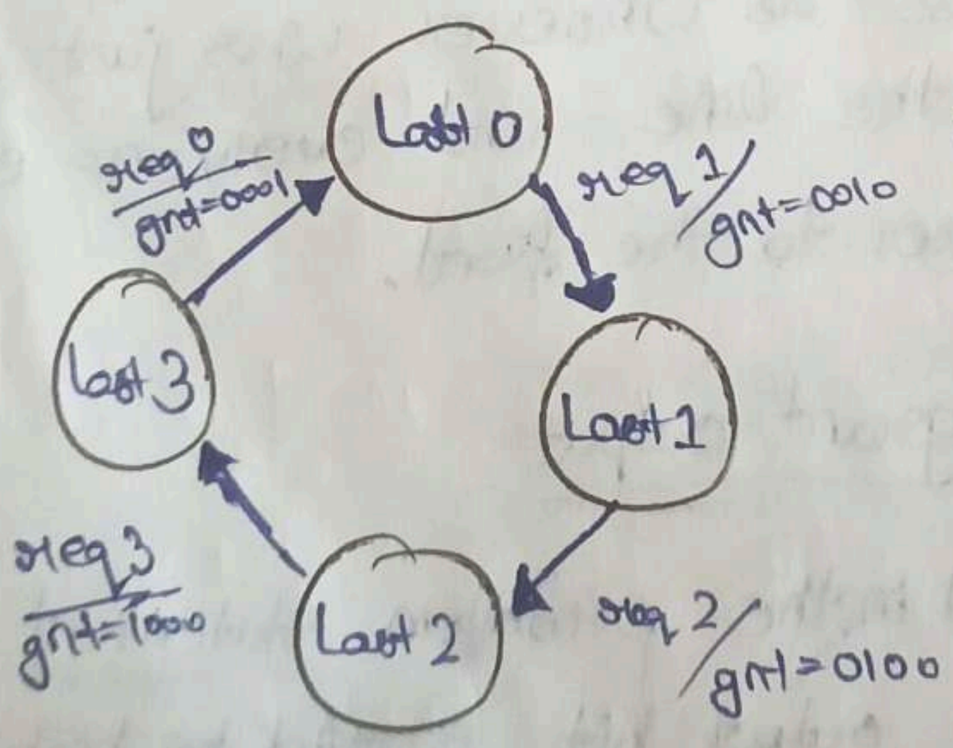
- The advantage of using one hot encoding for grant outputs is that we don't have to separately check if any two grant bits are high at the same time.
- One-hot ensures that atmost one bit can remain high.

Eg) If gnt = 4'b0110

⇓

There is a bug in our design !!

State Diagram



However, the real transition depend on "req".

If req = 0000 (no one is asking for request)
then gnt = 0000 and state stays at Last-X.
(self loop)

Ex) Last 2 (present state)

Check order = 3, 0, 1, 2

State Table based on example

Present State	Condition	gnt	Next state
Last 2	req 3 = 1	1000	Last 3
Last 2	req 3 = 0, req 0 = 1	0001	Last 0
Last 2	req 3 = 0, req 0 = 0, req 1 = 1	0010	Last 1
Last 2	~~req = 4'b0000~~ req = 4'b0100	0100	Last 2 (self loop
Last 2	req = 4'b0000	0000	Last 2 (self loop
Last 3	req = 4'b xxx1	0001	Last 0
Last 3	req = 4'b xx10	0010	Last 1
Last 3	req = 4'b x100	0100	Last 2
Last 3	req = 4'b 1000	1000	Last 3 (self loop)
Last 3	req = 4'b 0000	0000	Last 3 (self loop)
Last 0	req = 4'b xx1x	0010	Last 1
Last 0	req = 4'b x10x	0100	Last 2
Last 0	req = 4'b 100x	1000	Last 3
Last 0	req = 4'b 0001	0001	Last 0 (self loop)
Last 0	req = 4'b 0000	0000	Last 0 (self loop)
Last 1	req = 4'b x1xx	0100	Last 2
Last 1	req = 4'b 10xx	1000	Last 3
Last 1	req = 4'b 00x1	0001	Last 0
Last 1	req = 4'b 0010	0010	Last 1 (self loop)
Last 1	req = 4'b 0000	0010	Last 1 (self loop)

Code overview

Module declaration:

Module (clk, rst, req[3:0], gnt[3:0]; ~~state~~
and a 2bit State register);

For determining priority

for sequential logic,

always @(posedge clk) begin

if (rst) begin
state <= 2'b11;
end
else begin
case (gnt)

} (Reset condition)
⇓
initialized to last 3
so that next will be
zero.

4'b0001 : state <= 2'b00;
4'b0010 : state <= 2'b01;
4'b0100 : state <= 2'b10;
4'b1000 : state <= 2'b11;

} state will be
defined based
on who was
granted !!

Combinational logic

default grant will be zero

always@(*) begin gnt = 4'b0000;

we will use case statements on 'state' to
set the combinational priority order.

case (state)
2'b00: → Last 0 was assigned
2'b01: → Last 1 was assigned
2'b10: → Last 2 was assigned
2'b11: → Last 3 was assigned.